AR FLASHCARDS

Project Documentation & Setup Guide

Platform: Unity 6.x **AR framework:** Vuforia Engine 11.4.4

Project type: Marker-based augmented-reality learning application

This guide describes the project purpose, development setup, application workflow, testing and troubleshooting, Android build considerations, and GitHub repository maintenance.

Repository: github.com/Aryan-Gupta-9044/ARFlashcards

Prepared as a repository-friendly overview. Confirm exact scene names, target database contents, and device settings in Unity before a formal demonstration.

1. Project Overview

AR Flashcards is a Unity-based educational prototype intended to connect physical flashcards with digital 3D learning content. The learner points a supported camera at a target flashcard; when the target is recognized, the application can present associated content in an augmented-reality experience. A 3D viewer/fallback interface is also part of the current workflow, allowing content to be explored when the Vuforia camera experience is not active.

The current setup has included Vuforia Engine 11.4.4 and a flashcard interface with animal-card content, navigation controls, a 3D model viewer, and scan-progress text such as “Scanning… point the camera at a flashcard” and “Scanned 0/5.” Recognition depends on the target database, camera permissions, Vuforia configuration, and a compatible device.

Project goals

• Provide an interactive learning experience using physical image targets and digital content.

• Present 3D models and supporting interface elements in a flashcard-oriented workflow.

• Support validation in the Unity Editor before building to a mobile device.

• Keep source, package configuration, and setup notes organized for collaboration through GitHub.

2. Technology Stack

Technology	Purpose
Unity 6.x	Main editor, scenes, UI, assets, and build pipeline.
C#	Application scripts and behaviour logic.
Vuforia Engine 11.4.4	Image-target tracking and camera-based AR functionality.
Unity Package Manager	Package declarations and dependency resolution.
Git and GitHub	Source control, collaboration, and project distribution.
Test support files	The repository includes tools/test/VuforiaStubs.cs and tools/test/run_tests.sh; review their scope and dependenci

3. Application Workflow

• Launch: the Unity scene loads and initializes the interface.

• AR initialization: Vuforia initializes the AR camera and tracking system when the scene and device configuration support it.

• Target observation: the learner points the camera at an image target configured in the Vuforia target database.

• Target recognition: when detected, the corresponding AR content should appear according to the scene’s target behaviour.

• Learning interaction: the learner uses the available flashcard interface and 3D viewer controls.

• Fallback: if Vuforia does not start, the fallback viewer may still display 3D content; this does not prove camera tracking works.

> Important: fallback 3D viewing and a successful image-target AR session are different test outcomes. Validate them separately.

4. Development Environment Setup

Prerequisites

• A compatible Unity 6.x editor installation; use the exact project editor version if recorded.

• The project root containing Assets, Packages, and ProjectSettings.

• Vuforia Engine installed and configured in the Unity project.

• A valid Vuforia license key where required. Keep this key private.

• For Android testing, Unity Android Build Support and the required SDK/NDK and OpenJDK modules.

Open and verify the project

• Open Unity Hub and add/open the project root folder.

• Allow Unity to resolve packages and import assets; first import may take time.

• Open the main scene from Assets and check the Console for compile errors.

• Select the AR Camera and confirm that the expected Vuforia Behaviour/component is present.

• Confirm that the image target database is available, active, and contains the intended targets.

Vuforia checklist

• Check package installation in Package Manager.

• Verify the AR Camera and Vuforia components in the scene.

• Verify the license configuration without sharing the key publicly.

• Check camera permission and camera availability on the target device.

• Test with a clear, correctly printed target under good lighting.

5. Testing and Troubleshooting

Symptom	Checks
“Vuforia did not start”	Check license configuration, camera permission, device support, AR Camera configuration, and Console
“Vuforia Deinitialized”	Vuforia is not currently initialized; inspect initialization errors and lifecycle transitions.
Scanning, but count stays 0/5	Verify the target database, matching target image, camera focus, and lighting.
3D viewer works but AR does not	Treat viewer and tracking as separate paths; troubleshoot camera initialization and target recognition.
Setup Wizard GUI layout exception	Close the wizard if it throws a GUI layout error, then verify package installation and inspect the Console.
Android build fails	Check Android modules, Player Settings, package identifier, permissions, and the first actionable build er

Recommended test sequence

• Run the scene in Unity and verify that the interface loads without compile errors.

• Test each flashcard UI control and confirm content changes as intended.

• Test the 3D viewer/fallback independently.

• Build and run on a supported Android device; grant camera permission.

• Test each configured image target and confirm tracking detection.

• Record device model, Unity version, Vuforia version, and runtime errors when reporting defects.

6. Android Build Outline

• Install Android Build Support for the Unity editor if needed.

• Open File → Build Profiles (or the equivalent build settings) and select Android.

• Apply the platform switch/import step requested by Unity.

• Review package/application identifier, minimum API level, orientation, scripting backend, and project requirements in Player Settings.

• Verify camera permission and Vuforia settings; an Editor test does not confirm device camera behaviour.

• Connect a device with USB debugging enabled or build an APK/AAB for installation.

• Test installation, first launch, permission prompt, target recognition, UI, and fallback behaviour.

> Exact settings vary by Unity version, Vuforia configuration, and Android device. Resolve build errors before presenting the app as a working AR release.

7. GitHub Repository and Large Package Handling

Repository: Aryan-Gupta-9044/ARFlashcards

The initial push was rejected because Packages/com.ptc.vuforia.engine-11.4.4.tgz was approximately 131.89 MB, above GitHub's 100 MB per-file limit. Avoid committing this archive directly unless Git LFS is intentionally configured and suitable for the repository.

Recommended repository hygiene

• Commit the Unity source and configuration required to reproduce the project, especially Assets, Packages/manifest.json, Packages/packages-lock.json, and ProjectSettings.

• Ignore generated folders such as Library, Temp, Obj, Logs, and UserSettings.

• Ignore the local Vuforia .tgz archive if it is an installation artifact; document how to install Vuforia separately.

• Never commit license keys, access tokens, credentials, or machine-specific secrets.

• If the oversized file is already in an unpushed commit, remove it from Git's index and amend/rewrite that commit before pushing.

Example .gitignore entries

```
# Unity generated files
[Ll]ibrary/
[Tt]emp/
[Oo]bj/
[Ll]ogs/
UserSettings/

# Local Vuforia archive
Packages/com.ptc.vuforia.engine-11.4.4.tgz

# IDE generated files
.vs/
.idea/
*.csproj
*.sln
```

A common repair for an unpushed rejected commit is to untrack the archive, add the ignore rule, amend the latest commit, and push again. Check Git status and commit history first.

8. Suggested Repository Contents

Path	Purpose
Assets/	Scenes, scripts, prefabs, models, materials, textures, UI, and target-related assets.
Packages/manifest.json	Declares Unity package dependencies.
Packages/packages-lock.json	Records resolved package versions when maintained by the project.
ProjectSettings/	Unity project configuration and editor/player settings.
tools/test/VuforiaStubs.cs	Test-support stub file present in the repository; review its intended test usage.
tools/test/run_tests.sh	Test runner script present in the repository; review dependencies and instructions.
README.md	Quick summary, requirements, run instructions, troubleshooting, and licensing notes.
AR_Flashcards_Project_Documentation.pdf	Expanded project overview and setup guide.

9. Demonstration Checklist

• Open the project in the intended Unity editor version.

• Show the scene and AR Camera/Vuforia configuration without exposing the license key.

• Run the app and demonstrate the flashcard interface.

• Demonstrate the 3D viewer/fallback path.

• On a supported device, demonstrate camera initialization and target detection if working.

• Explain the difference between fallback 3D viewing and confirmed AR tracking.

• Show the GitHub repository and explain intentionally excluded local/generated files.

• Clearly state any known limitations or setup requirements.

10. Scope and Verification Notes

This guide is based on project details and console/output snippets available in the conversation. It is a practical overview, not a code audit of every scene and script. Confirm exact scene names, the complete target list, the mapping between individual cards and models, device compatibility, and final Android build status directly in Unity.

Before publishing, verify that the repository contains no private keys or assets that cannot be redistributed. Add screenshots and a demonstration video to the README if permitted by the submission requirements.

Appendix: Quick Git Commands

Run from the Unity project root in PowerShell after checking the branch and remote:

```
git status
git add .gitignore
git rm --cached Packages/com.ptc.vuforia.engine-11.4.4.tgz
git commit --amend --no-edit
git push -u origin main
```

If the commit has already been shared with collaborators, coordinate before rewriting history. If the push still fails, inspect the exact file path reported by GitHub; another oversized file may remain in the commit history.